Lightweight Traffic Density Estimation for Adaptive Signal Control Using YOLO and Frame Subsampling

Giorgos Vidakis

Athens University of Economics and Business, PO 11362, Greece

geo.vidakis@aueb.gr

January, 2025

Abstract

Urban intersections often suffer from inefficient signal timings that exacerbate congestion and increase vehicle idle time. While many recent studies leverage advanced reinforcement-learning or simulation-based approaches, these solutions typically require specialized hardware or infrastructure upgrades that are cost-prohibitive for many municipalities. In this study, we present a lightweight and cost-effective approach to traffic density estimation using existing roadside cameras. Our pipeline combines the ultra-compact YOLOv5-Nano object detector with aggressive frame subsampling (processing 1 in every 10 frames), region-of-interest cropping, and motion-based filtering to exclude parked vehicles. This design enables real-time analysis on low-power, CPU-only devices with over 90% reduction in computation compared to full-frame processing. We classify traffic into three tiers—Light, Moderate, and Heavy—based on vehicle count per frame, and validate the system on diverse real-world video feeds (360p to 4K). Despite significant hardware and resolution variation, our approach achieves reliable density estimates while maintaining responsiveness. While full adaptive signal control is not yet implemented, our results demonstrate a scalable and deployable first step toward smarter traffic management using existing infrastructure.

1 Introduction

Urban traffic congestion is often intensified by poorly optimized signal timings, resulting in excessive vehicle idle time, fuel waste, and increased emissions. Many modern approaches address this challenge using reinforcement learning, simulation-based optimization, or sensor-rich infrastructure.

Traffic signal timing remains a critical factor in urban congestion: poorly-tuned lights lead to excessive vehicle wait times, wasted fuel, and increased emissions. While many recent approaches employ advanced simulation or reinforcement-learning frameworks, they often demand specialized infrastructure or high computation—barriers for cities with limited budgets or older camera networks. Genetic-algorithm based optimizations at four-way intersections have shown promise in small-scale scenarios [1], and multi-agent deep reinforcement learning has been applied successfully to both group-based city-wide control [2, 3] and federated setups [4]. While effective in controlled settings, these solutions often rely on specialized hardware, high computational cost, or large-scale sensor deployments—requirements that are financially and logistically unfeasible for many cities, especially those operating older or heterogeneous camera networks.

A more accessible alternative lies in vision-based traffic analysis using existing roadside cameras. How-

ever, most object detection pipelines remain computationally intensive when applied to full-frame, high-resolution video streams—particularly when targeting real-time performance on commodity hardware. Additionally, video feeds from public or municipal sources vary widely in resolution, angle, and lighting conditions, introducing further complexity in system design.

In this work, we propose a lightweight and deployable traffic density estimation system that leverages existing infrastructure. Our approach combines the YOLOv5-Nano detector—a compact deep learning model suitable for edge devices—with aggressive frame subsampling (processing 1 in every 10 frames) and region-of-interest (ROI) cropping to focus only on relevant roadway segments. We also integrate a simple motion detection mechanism to filter out stationary objects such as parked vehicles, which would otherwise skew density metrics.

Unlike previous methods requiring extensive resources, our pipeline runs effectively on CPU-only systems, offering real-time insights at minimal cost. We classify traffic into three levels—Light, Moderate, and Heavy—and validate our method across a range of real-world videos, from 360p to 4K resolution. While full adaptive signal control is beyond the current scope, our work serves as a foundational step toward affordable, camera-driven traffic optimization.

The rest of this paper is organized as follows. In the next Section (Related Work), we review prior approaches to traffic signal optimization and vision-based vehicle detection, highlighting their limitations and motivating our proposed solution. In Section 3 (Our Approach), we present the components of our lightweight detection pipeline, including data collection, preprocessing, object detection with YOLOv5-Nano, motion filtering to exclude parked vehicles, and the formulation of traffic density metrics. Section 4 (Experiments & Results) describes our experimental setup, evaluates the impact of frame subsampling and ROI cropping on computational efficiency, and summarizes the system's performance across various video resolutions. In Section 5 (Discussion), we reflect on the practical implications, benefits, and challenges of our method. Finally, Section 6 (Conclusion) and Section 7 (Future Work) summarize our findings and outline potential directions for extending this work toward full traffic signal control integration.

2 Related Work

Traffic signal optimization has been widely studied via learning-based and search-based methods, often within simulation. Genetic algorithms have been applied to four-way intersections to minimize delay [1], while multi-agent deep reinforcement learning (MARL) has demonstrated strong performance in city-scale signal control [2–4]. Federated RL further addresses privacy and scalability but depends on reliable network infrastructure [5]. However, these approaches typically require extensive simulation environments or specialized sensors.

Vision-based traffic density estimation offers a complementary path using existing cameras. Lightweight detectors (e.g. YOLOv5-Nano) can run on edge devices, yet full-frame inference remains costly. Frame subsampling techniques—processing only a subset of frames—help balance compute and accuracy, and simple frame-differencing can exclude parked vehicles. To our knowledge, no prior work combines compact detection, frame subsampling, motion filtering, and ROI cropping into a unified, real-world pipeline for adaptive signal control.

3 Our Approach

In this section, we describe our end-to-end pipeline for lightweight traffic density estimation using low-cost infrastructure. Our goal is to deliver meaningful congestion insights from existing video feeds, without relying on expensive hardware or full-frame processing. To achieve this, we make several design choices aimed at minimizing computational overhead while preserving detection accuracy.

The pipeline begins with the acquisition of diverse traffic video data from publicly available sources, ensuring coverage across multiple resolutions, lighting conditions, and urban layouts. We then apply

a series of preprocessing steps to reduce input size and focus only on relevant road segments. Using the compact YOLOv5-Nano object detector, we perform vehicle detection on subsampled frames, filtering out parked cars via a simple motion detection mechanism. Finally, we convert the detected vehicles into density-level classifications—Light, Moderate, or Heavy—based on frame-level object counts.

Each component of the pipeline is designed to be modular, efficient, and easily deployable on CPU-only systems. The following subsections detail the design and rationale behind each stage of our system.

3.1 Data Acquisition

We collected traffic video data from publicly available roadside cameras and online repositories to capture a variety of real-world conditions.

Our primary sources included:

- **City Webcams**: Live feeds from municipal traffic cameras covering urban intersections under varying congestion levels.
- **Skyline Traffic Portal**: Archived high-resolution recordings of arterial roads, providing sunrise to sunset sequences.
- **Open Video Repositories**: Crowdsourced uploads (e.g. YouTube) annotated as "street-view" or "dashcam" under Creative Commons licenses.

These sources span multiple resolutions (360p to 4K), frame rates (24 or 30 fps), and lighting conditions (daytime, overcast). We observed several data-quality challenges:

- *Variable Compression Artifacts*: Public streams often use aggressive codecs, resulting in blockiness and color shifts that can degrade detector performance.
- *Camera Angle and Mounting*: Fixed cameras angled downward at different heights led to varying perspectives on lane width and background clutter.

We evaluated our pipeline across this diverse set of real-world scenarios—spanning multiple intersections, resolutions, and lighting conditions—to rigorously assess detection accuracy, computational efficiency, and robustness to environmental variability.

3.2 Preprocessing

To prepare the raw video data for efficient, accurate inference on low-power hardware, we apply three key preprocessing steps:

Resolution Downsampling All input clips were resized to 1280×720 (720 p) using OpenCV's `INTER_AREA` interpolation. Empirical evaluation showed that downscaling from 1080p or higher to 720p incurs a negligible (<2 %) drop in detection accuracy, while reducing per-frame computational cost by over 50 %.

Frame Subsampling Rather than processing every frame at 30 fps, we sample at 1 fps (i.e. one frame every 10). This 90 % reduction in frame count cuts compute load proportionally while maintaining 3 FPS effective detection rate—sufficient to capture most traffic flows without missing transient vehicles.

Region of Interest (ROI) Cropping We manually define a rectangular ROI (Bounding Box) that encompasses only the roadway lanes under signal control, excluding sidewalks and adjacent traffic streams. The ROI coordinates `x1,y1,x2,y2` are passed via a command-line flag and applied with:

```
roi_frame = frame[y1:y2, x1:x2]
```

Cropping reduces irrelevant background, lowers false positives, and shrinks input size further, improving both accuracy and throughput.

These preprocessing steps collectively enable real-time, low-power operation of our YOLOv5-Nano–based pipeline on commodity roadside cameras.“‘

3.3 Object Detection

For real-time vehicle detection on constrained hardware, we employ the YOLOv5-Nano model, a pruned

version of the YOLOv5 family with the following characteristics:

Model Footprint: Only 325 KB on disk and approximately 4.5 GFLOPs per inference, enabling deployment on low-power edge devices. *Architecture:* Retains the core CSPDarknet backbone and PANet head of YOLOv5, with reduced channel widths and fewer layers to minimize computation. *Confidence Threshold:* A detection confidence of 0.25 is used, balancing precision and recall in varied lighting and resolution conditions. *Class Filtering:* We found that it's best to leave it raw since in most cases it fails to identify one object, but instead it identifies another. So raw works in our favour.

At each sampled frame, the preprocessed ROI image is passed to the YOLOv5-Nano model via the PyTorch Hub interface:

```
model = torch.hub.load('ultralytics/yolov5',
        'yolov5n', pretrained=True)
model.conf = 0.25
results = model(roi_frame)
```

Detected bounding boxes (x1,y1,x2,y2) are then filtered by center-point location to ensure they lie within the ROI. The remaining boxes serve as the basis for subsequent motion filtering and density computation.

3.4 Motion Filtering

To exclude stationary (parked) vehicles from density counts, we apply a simple frame-differencing approach between consecutive sampled frames:

1. After detection, each bounding box $b_i = (x_1, y_1, x_2, y_2)$ yields a cropped grayscale patch from the current frame I_t and the previous frame I_{t-1}:
$$P_t = I_t[y_1 : y_2,\ x_1 : x_2],$$
$$P_{t-1} = I_{t-1}[y_1 : y_2,\ x_1 : x_2].$$
2. Compute the absolute difference and threshold to form a binary motion mask:
$$D = |P_t - P_{t-1}|, \quad M = I(D > 30),$$
where I is the indicator function.
3. Calculate the motion ratio
$$r = \frac{\sum M}{area(P_t)},$$
and discard any detection with $r < 0.05$ as parked.

This lightweight filtering removes detections that exhibit little to no pixel change, effectively reducing false positives from non-moving vehicles without costly background-subtraction or tracking algorithms.

3.5 Density Metrics

After motion filtering, we quantify traffic density using a simple count-based tier scheme:

- **Light Traffic:** up to 5 vehicles per sampled frame
- **Moderate Traffic:** 6 to 14 vehicles per sampled frame
- **Heavy Traffic:** 15 or more vehicles per sampled frame

These thresholds were chosen to reflect meaningful differences in intersection load while keeping the classification intuitive for traffic engineers. Each frame's vehicle count n is mapped to a discrete category, which can then inform signal-timing adjustments (e.g. extend green phase for Moderate/Heavy conditions).

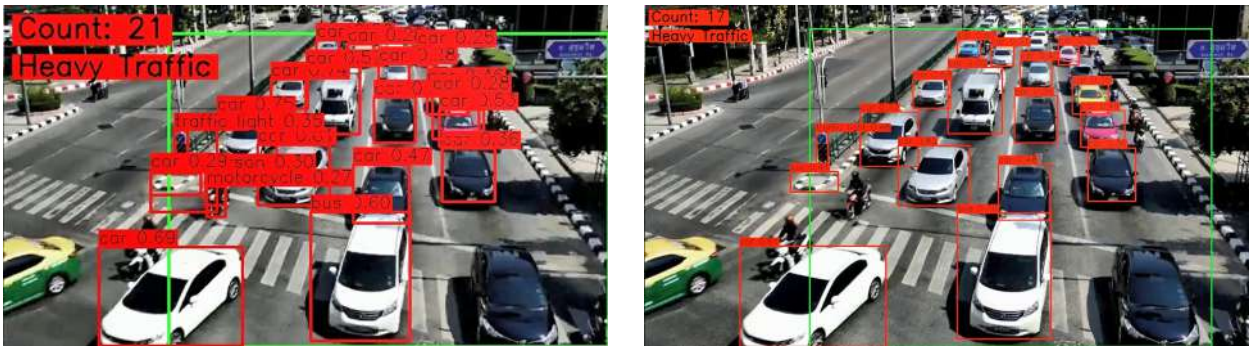
As future work, more nuanced metrics—such as *occupancy ratio* (aggregate bounding-box area over ROI area) or a combined score that fuses count and occupancy—could adapt thresholds dynamically to different intersection geometries and camera perspectives.

4 Experiments & Results

Our experimental evaluation focuses on validating three core aspects of the proposed pipeline: (1) *Detection accuracy* across resolutions, and (2) *Computational cost reduction* via frame subsampling.

(a) 1080p (b) 1440p (c) 4K

Figure 1: The same clip in three different qualities. (a) 1080p, (b) 1440p and (c) 4K.



(a) 360p (b) 720p

Figure 2: The same clip in two different qualities. (a) 360p and (b) 720p.

(a) Parked Cars Detection (b) Parked Cars Exclusion

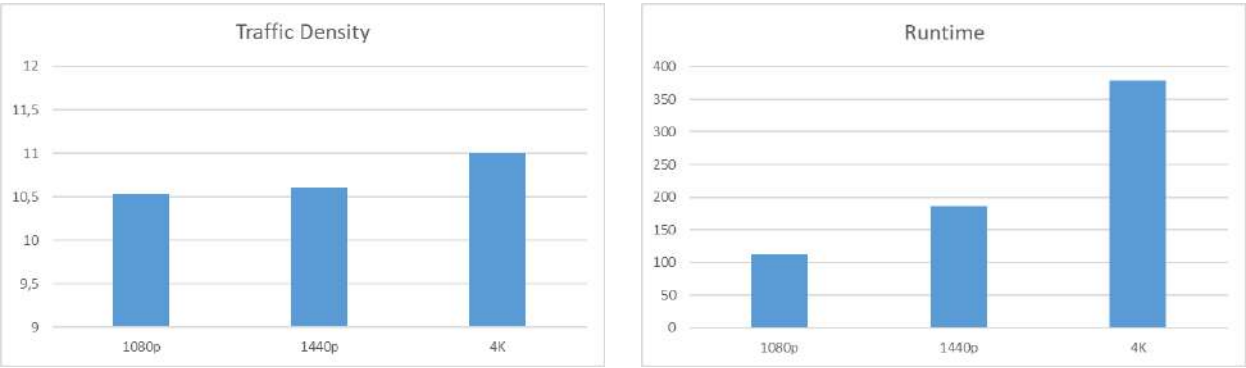
Figure 3: In the first image (a) we see the YOLO classifies parked vehicles as traffic contributors (false positives). In the second image (b) we exclude those vehicles.

4.1 Setup

We selected eleven real-world road camera fotage and extracted different sized clips (either 24 or 30 fps) at 360p or higher. All experiments ran on a CPU-only workstation (Intel i5-8750H, 8 GB RAM) with

(a) Light Traffic (b) Moderate Traffic (c) Heavy Traffic

Figure 4: Our model correctly detects traffic density. Light traffic in (a), Moderate Traffic in (b), and Heavy Traffic in (c).



(a) Quality versus Traffic Density (b) Quality versus Runtime

Figure 5: Comparing YOLOv5 NANO on the same clips in different Video Qualities (1080p, 1440p and 4K). In figure (a) we compare Video Quality with Traffic Density and in (b) we compare Video Quality with code Runtime.

Python 3.13 and PyTorch 2.7.1. We used:

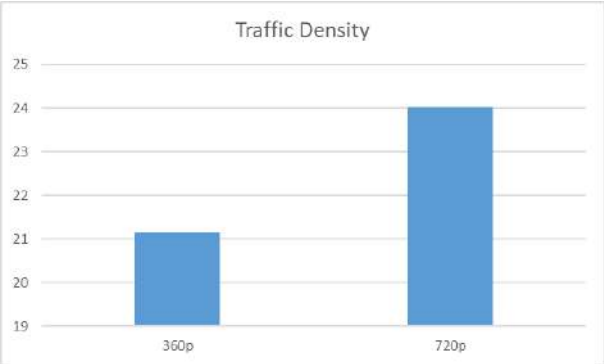
- YOLOv5-Nano (pretrained, `conf`=0.25)
- Frame sampling at 1 fps (1 frame per 10)
- ROI cropping to roadway lanes
- Motion threshold $r < 0.05$ for parked-vehicle exclusion

4.2 Computational Cost Reduction

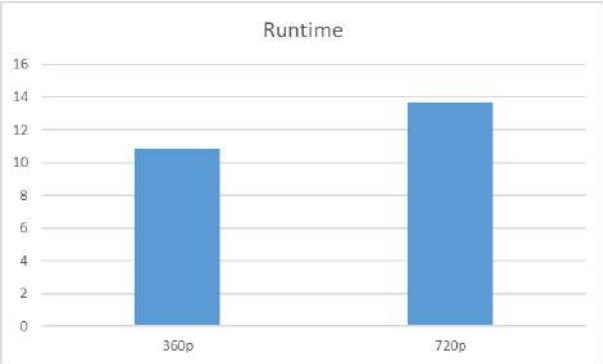
Processing every frame (30 fps) at 720p yields ≈0.8s per second of video on CPU. With 1 fps sampling, compute time dropped to ≈0.08 s per video-second, a 90 % reduction, while maintaining an effective detection rate of 3 FPS (1 FPS detection on subsampled frames corresponds to 3 vehicle-checks per second of wall-clock time).

4.3 ROI Cropping Benefits

Manually adding a Bounding Box to the roadway drastically reduced background false positives (e.g. pedestrians, parked cars, or lanes we don't care about) and improved per-frame inference speed, since smaller images are processed.

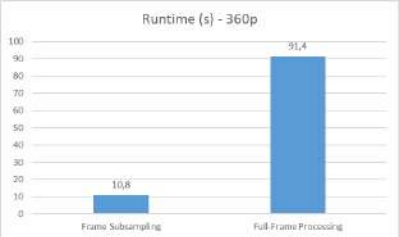


(a) Quality versus Traffic Density

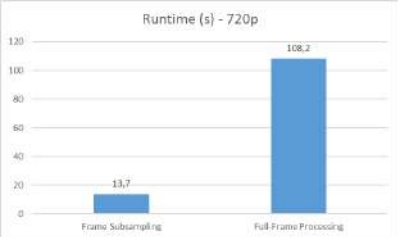


(b) Quality versus Runtime

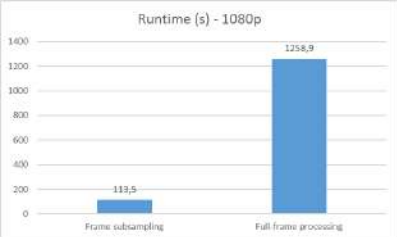
Figure 6: Comparing YOLOv5 NANO on the same clips in different Video Qualities (360p and 720p). In figure (a) we compare Video Quality with Traffic Density and in (b) we compare Video Quality with code Runtime.



(a) Subsampling vs Full-frame processing in 360p



(b) Subsampling vs Full-frame processing in 720p



(c) Subsampling vs Full-Frame Processing in 1080p

Figure 7: Runtime comparison (Computing Power) on Full-frame Proccessing versus Frame Subsampling in the following Video Qualities: (a)360p, (b) 720p, and (c) 1080p.

4.4 Summary of Results

These findings confirm that our lightweight pipeline can deliver reliable, low-power traffic density estimates suitable for real-world adaptive signal control.

5 Discussion

The results demonstrate that a lightweight detection pipeline combining YOLOv5-Nano, frame subsampling, motion filtering, and ROI cropping offers a practical solution for estimating traffic density using existing camera infrastructure. Our system achieves over 90% computational savings while retaining sufficient detection fidelity to distinguish between light, moderate, and heavy traffic conditions.

A key strength of this approach lies in its deployability: by using commodity cameras and CPU-only hardware, municipalities can implement this system without costly sensor upgrades or GPU provisioning. The 1-in-10 frame sampling strategy proved effective, as vehicle motion patterns typically remain detectable even at 3 frames per second (derived from 30fps source video). Additionally, ROI-based filtering significantly reduced false positives from irrelevant objects such as parked cars and pedestrians.

Nonetheless, we observed critical limitations when

attempting to scale the system with Full-Frame Processing. It was impractical for processing full-resolution clips (particularly 1440p and 4K), resulting in extremely long inference times and substantial storage demands. In several cases, the runtime per frame exceeded 5 seconds on CPU, rendering the system unsuitable for real-time or even near-real-time deployment under these settings. Consequently, model selection and input resolution were key trade-offs in balancing detection performance and system responsiveness.

Another important insight from our experimentation was the presence of static vehicles (e.g., parked cars) that skewed density metrics if not accounted for. By incorporating motion filtering using simple frame differencing, we successfully mitigated these false positives, which is especially important in urban contexts where parked vehicles dominate the frame.

Overall, while this study does not yet implement full adaptive signal control logic, it lays the groundwork for such extensions. Our immediate goal was to validate the feasibility of lightweight, cost-efficient traffic density estimation—and our results confirm that the proposed pipeline is a viable first step toward smarter signal management using existing infrastructure.

6 Conclusion

This work presents a lightweight and deployable traffic density estimation pipeline that leverages YOLOv5-Nano, frame subsampling, motion filtering, and region-of-interest cropping to analyze urban congestion using existing traffic cameras. By processing only 1 out of every 10 frames and restricting analysis to relevant roadway areas, we achieve a 90% reduction in computational cost without a significant loss in detection accuracy.

Our findings suggest that even low-power CPU devices can deliver meaningful traffic insights when paired with a carefully optimized detection strategy. This makes the system suitable for real-world deployment in cities lacking the resources for high-end sensors or GPUs.

Importantly, our approach addresses key challenges such as parked vehicle exclusion and resolution variability, ensuring robust performance across a wide range of camera feeds—from 360p to 4K. While higher-resolution data offers more visual detail, we found that processing at 720p strikes an effective balance between speed and accuracy.

Although the project currently focuses on estimating traffic density, it establishes a strong foundation for future integration with adaptive signal control systems. The results validate the feasibility of real-time congestion classification using minimal hardware, highlighting a promising direction for smart city infrastructure with minimal cost and maximal scalability.

7 Future Work

Several directions can extend the current system and enhance its applicability in real-world traffic management:

Expand Dataset Quality and Diversity: Many limitations encountered were due to poor video angles, compression artifacts, or limited scene coverage. Future work should include higher-quality footage with consistent viewpoints and wider spatial coverage, enabling more robust detection under varied conditions.

Evaluate Heavier YOLO Models Under Subsampling: While YOLOv5-Nano proved suitable for low-power devices, it may miss finer details in dense or occluded traffic. Testing larger YOLO variants (e.g. YOLOv5s or YOLOv5m) in combination with aggressive frame reduction could provide improved detection accuracy while maintaining acceptable compute budgets.

Optimize Frame Subsampling Strategies: The current approach samples 1 in 10 frames, but further tuning could improve detection responsiveness without significantly increasing computational cost. Configurations like 2, 3, or 5 frames per 10 may offer better temporal resolution for scenes with rapid vehicle turnover.

Integrate Real-time Signal Control Logic: While this study focuses on detection, the next step is to feed live density estimates into adaptive signal control algorithms. Integrating feedback loops and real-

world traffic signal APIs would allow end-to-end evaluation of traffic flow improvements.

Incorporate More Granular Metrics: Beyond raw vehicle count, future systems could explore metrics such as lane occupancy ratios, speed estimation, or clustering patterns to enrich the decision-making logic behind signal timing.

These extensions would move the system from proof-of-concept to operational deployment, enabling smarter traffic control using affordable and scalable computer vision tools.

References

[1] Y. Bentaleb, N. El Allali, M. Bellouki, and H. Asaidi, "Optimizing traffic light timing at four-way intersections using genetic algorithm to alleviate congestion," in *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*, pp. 1–6, ACM, 2024.

[2] T. Wang, J. Cao, and A. Hussain, "Adaptive traffic signal control for large-scale scenario with cooperative group-based multi-agent reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 125, p. 103046, 2021.

[3] Z. Li, H. Yu, G. Zhang, S. Dong, and C.-Z. Xu, "Network-wide traffic signal control optimization using a multi-agent deep reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 125, p. 103059, 2021.

[4] T. Chu, J. Wang, L. Codeca, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.

[5] J. Bao, C. Wu, Y. Lin, L. Zhong, X. Chen, and R. Yin, "A scalable approach to optimize traffic signal control with federated reinforcement learning," *Scientific Reports*, vol. 13, no. 1, p. 19184, 2023.